

Game Proposal: StackOverflow

CPSC 427 – Video Game Programming

Team: Team ASIMOV

Thomas Gray 59527523

Dean Yang 67057695

Sky Huang 38929873

Amanda Tian 49593320

Vincent Gilbert 67092841

Story:

Briefly describe the overall game structure with a possible background story or motivation. Focus on the gameplay elements of the game over the background story.

The game begins with the player character, a robot, waking up in a corner of a lab. When the robot first wakes up, it is greeted by a call from the scientist via a dialogue box. He introduces himself as a scientist who has been trapped in a lab overrun with rogue robots. Although he's unable to escape himself, he's glad he's found a robot that hasn't gone rogue and offers to remotely guide the robot to the lab exit. However, in order to traverse through the lab, he warns that the robot must defeat all enemy robots in each room before gaining clearance to move onto the next room. He then reassures the robot that it's equipped with the tools to do this, explaining to the player how to move, dash, shoot bullets, and make use of the bullet stacking mechanic. Finally, the scientist ends the call, directing the robot to navigate through the first area of the lab. Once the robot reaches that checkpoint (i.e. after defeating the first boss), he'll contact the robot again for more directions.

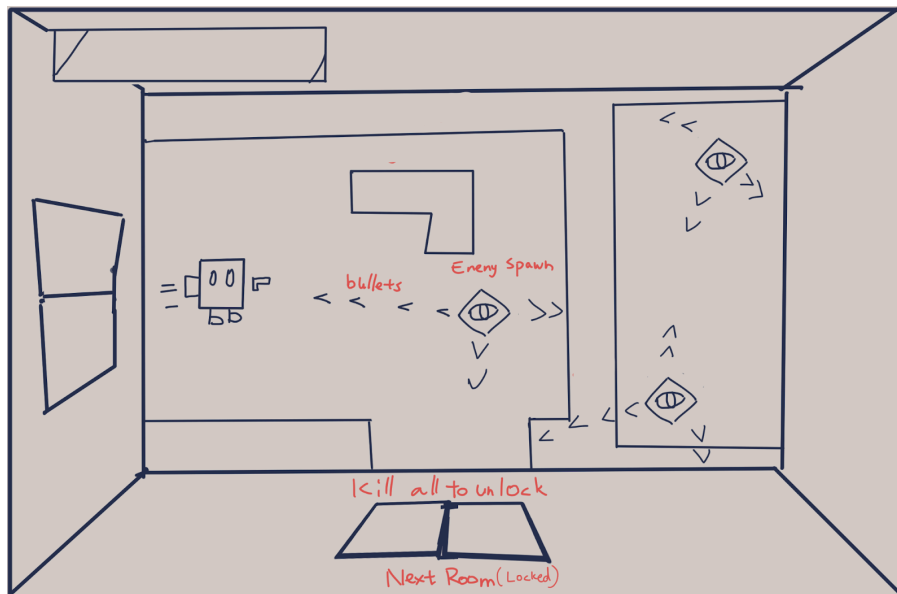
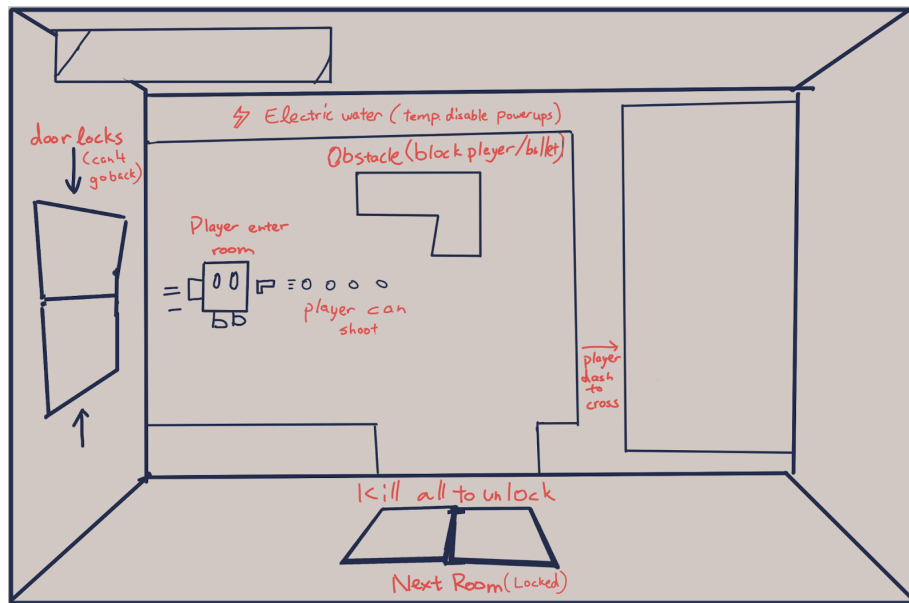
As the player traverses through the lab, they are encouraged to interact with objects (such as lab equipment, lab notes, etc.) and read about them to learn more about the lab, the origins of the robot, and the true intentions of the scientist.

Once the player reaches the lab exit at the very end, they will be blocked off by the scientist. He reveals that he had been testing the robot's combat ability by pitting it against combat robots. Although he's proud that the robot made it to the end, he declares himself as the final test before the robot is deemed strong enough to survive the world outside, which is overrun by even more combat robots. The player must then fight the scientist as the final boss.

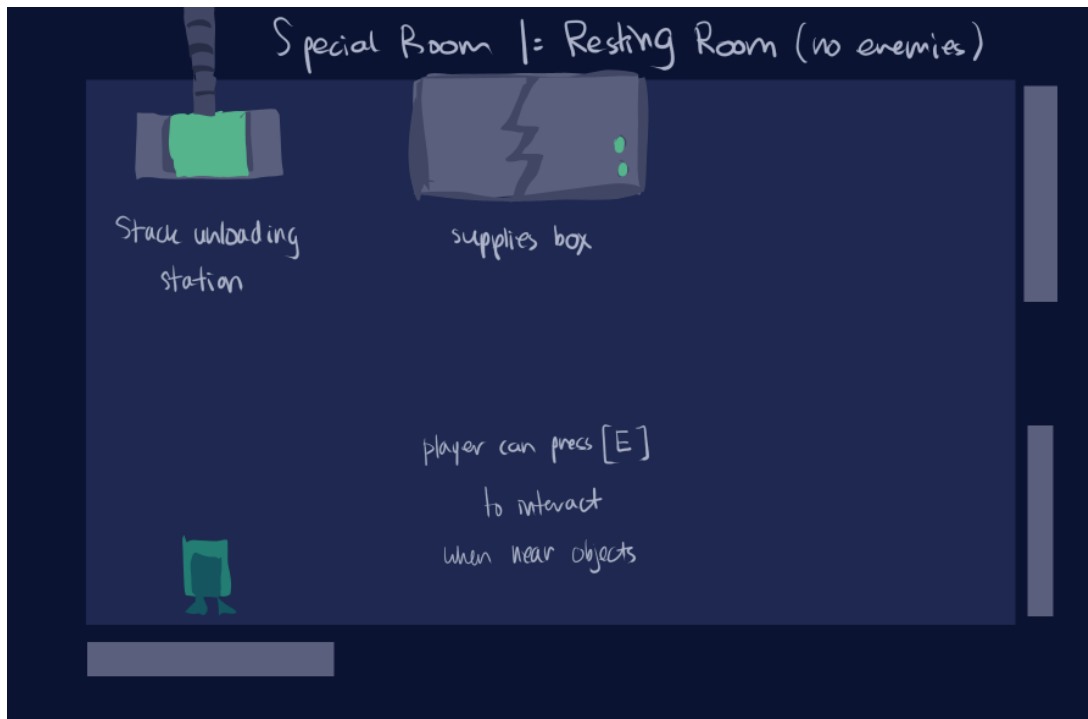
Once the fight is over and the scientist is weakened, the player can either: a) deal the last blow and kill him, then take the key from him to leave the lab or b) forgive him and convince him to leave the lab together with the robot by bringing the relevant story items.

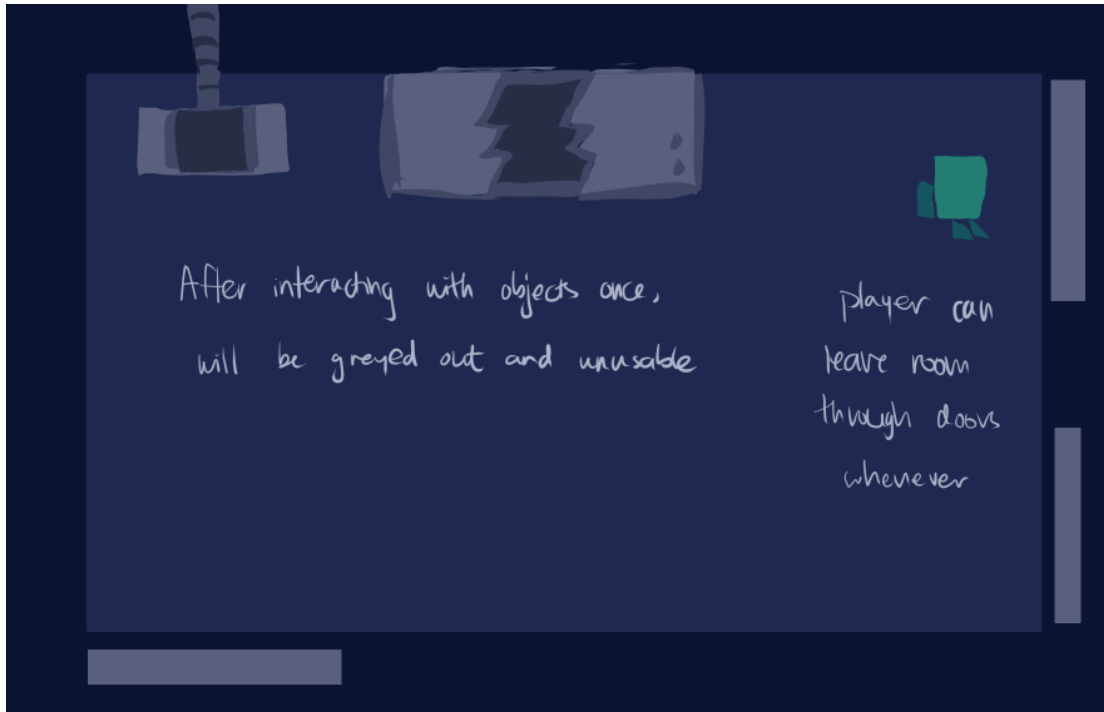
Scenes:

Produce basic, yet descriptive, sketches of the major game states (screens or scenes). These should be consistent with the game design elements, and help you assess the amount of work to be done. These should clearly show how players will interact with the game and what the outcomes of their interactions will be. For example, jumping onto platforms, shooting projectiles, enemy pathfinding or 'seeing' the player. This section is meant to demonstrate how the game will play and feeds into the technical and advanced technical element sections below. If taking inspiration from other games, you can include annotated screenshots that capture the gameplay elements you are planning to copy.



Additional rooms:





Technical Elements:

Identify how the game satisfies the core technical requirements: rendering; geometric/sprite/other assets; 2D geometry manipulation (transformation, collisions, etc.); gameplay logic/AI, physics.

The full game will feature 3 main regions with 1 boss for each region. Each region will also have its own enemies and music. For the scope of this course, we plan to start with 2 regions, and add more if time permits it (or remove the second region if time does not allow).

Assets (geometry, sprites, audio, etc.)

- We will make most visual assets ourselves, including sprites of the player, enemies, NPCs, UI, rooms and so on. We will aim to get assets for the player, enemies and NPCs. If we're short on time, we will get assets online, such as kenny.nl.
- Get audio from online open-source + design some of our own sounds
- Dark background, bright entities
- **Design Constraints:**

- Must adhere to the Amanda's Palette



-
- Must be TEXTMODE (lvlvl.com)
 - Vector mode, Modular shapes
- Must be proportionate to player sprite, 8 tiles wide

Rendering (visual effects, parallaxing, etc.)

- shaders for visual effects, such as CRT scan lines, bloom, and glow effects
- sprites implementation
 - Method 1:
 - import sprite (as vector image) into Blender to get a shape that covers sprite
 - export vertex map from Blender into OpenGL
 - load in vector image as texture
 - Method 2:
 - every vertex is a box that just has texture that shows shape, indicating transparency

2D geometry manipulation (transformation, collisions, etc.)

- Collision detection between player and enemy (contact damage)
- Collision detection between player and bullet
- Collision detection between player/bullets/enemies and walls of room
- Collision shapes:
 - Circle collision (radius + center)
 - Line segment collision (start + end)
 - Axis-aligned bounding box (AABB) collision (top right corner + bottom left corner for room)
 - note: no enemy-enemy collision

Gameplay logic (world interaction, character interaction, player controls, etc.)

- Player controls movement and combat of main character
 - dashes: start with one dash, has cooldown before can dash again
- Rooms
 - rooms are premade and are procedurally linked
 - each door to next room has screen that indicates what the next room will have, for player agency
 - each region has a specific theme (region of lab owned by different scientist), so rooms will look different
 - room types
 - standard enemy room
 - boss room
 - rooms with lore-related readables/interactables
 - resting room (unload stack)
- Different types of bullets that player can collect and put onto stack, buff/debuff player
 - bullets are classified by colour and shape
 - shape convey category
 - round (affect player self) - move speed, stack size, dash charge, dash cdr
 - sharp (affect player bullets) - bullet speed, fire rate, dmg, bounce, piercing, tracking
 - colors convey type within each category

- red - damage
 - yellow - speed
 - green - fire rate
 - blue - move speed
- bullet abilities (special bullets can give these abilities to player)
 - bouncing bullets. multiple counts will yield more bounces
 - piercing bullets
 - tracking bullets. multiple counts will make tracking “better” (increase tracking radius, or increase bullet turn speed)
- traps in certain rooms (optional)
 - lasers that turn on/off , when hit by laser player will get 1 no effect bullet in stack
 - gun in wall that periodically shoot bullets (will give bullets)
 - water puddle with electricity (like lightning bullets*)
- dialogue system for speaking with NPCs
- player can interact with objects to pull up dialogue description of objects (eg: readables)

AI (pathfinding, entity behavior, etc.)

- Enemies AI / Designs
 - Low Level:
 - Stationary enemy that shoots every 0.5 seconds towards the player
 - Fast moving but random direction, shoots 5 bullets in all directions when it hits a wall (low range)
 - Slow moving toward player, shotgun like attack infrequently
 - Medium Level:
 - Stationary enemy that shoots 4 bullets that home on where the player was when shot (arcing shot)
 - Stationary enemy that shoots 2 slow moving homing bullets
 - Worm enemy that shoots broadside, moves around the room next to the wall
 - Small BOID swarm, chooses new entity to fly towards (enemies/player) every 3 seconds
 - Enemy that moves around, shoots 2 arcing bullets and 1 big bullet towards player
 - Enemy that splits into 2 smaller enemies on death
 - Enemy that heals other enemies
 - Enemy that shoots lightning bullet*
 - High Level:
 - Random moving enemy that shoots 3 rounds of all direction bullets, with angle offsets
 - Spinning laser enemy in the middle of the room
 - Enemies that get hit can teleport toward random place
 - *Lightning bullets mess with the stack, can remove element, change it to something else, etc
- Boss AI / Designs
 - Boss that uses enemy stats in phase 1, player stats in phase 2 (may reserve mechanic for final boss)
 - duo boss: if kill first one, second will get angrier

- consider a laser beam linking them as an attack
 - consider resurrection mechanic - if 1 is killed , should kill another fast before 1 resurrect with 25% hp
- hivemind/swarm boss (BOIDS)
 - swarm boss can summons mini enemies , where the mini enemies can combine each other to make it more powerful
 - mini enemies can combine back to the main BOIDS and heal the boss
 - main BOIDS can shoot in all direction and spinning for 8 seconds
- Worm boss (destroying middle segments makes the boss split into distinct enemies) (implementation: head is “leader”, segments follow leader)
- 3 bosses and then a scientist
 - start with 1 region 1 boss, add additional bosses if time allows
- Stack extension for finishing bosses rather than sparing them

Physics (entity interactions, particle effects, kinematics, etc.)

- Bullets physics
 - bouncing bullets: these bullets will bounce off of walls and obstacles but do not bounce when hit enemies or a player. The number of times a bullet can bounce before it expires upon impact can vary (somewhere between 1 to 3).
 - homing bullets: these bullets will home on the player/enemy. After bullet is shot, it will seek out the nearest entity in its homing radius (radius around bullet) and start tracking that entity. Homing bullets will fly forwards until it found entity in its radius. Alternative: before being shot, bullet will lock in on nearest entity position, and then fly towards that fixed position when fired.
- add particle effects for bullet collisions
- player acceleration
 - player will dash 3 squares towards to the designated using W A S D as direction for a short time

Sound (effects, music, etc.)

- General direction of sound effects: robot-y tech sounds
 - when special bullets are shot
 - when player / enemies get hit
 - when player walks / dashes
 - signal enemy about to attack
- Music
 - Each region and bosses has different music theme
 - For 3 regions, 4 bosses and the title screen, we plan to have 8 tracks
 - for time constraint of course, stick to 1-2 region, 2 bosses (4-5 tracks)

Advanced Technical Elements:

List the more advanced and additional technical elements you intend to include in the game prioritized on likelihood of inclusion. Describe the impact on the gameplay in the event of skipping each of the features and propose an alternative.

Advanced Enemy AI

BOIDs as an enemy AI. Inclusion would have a visual impact, and a unique gameplay experience. Skipping this feature would be acceptable if the replacement AI was interesting enough, though a BOID system would be adaptable to many different enemies by tweaking parameters.

Worm-like enemies/bullets, entities that follow behind a leader enemy. Similar consideration to BOIDs

Dynamic Assets

Dynamic assets, e.g. stack visually indicated on player sprite. Low priority, purely eye-candy as the stack would be hard to read compared to the constant UI stack. Also may not be feasible depending on final stack size

More room variants

Special event rooms (eg: parkour, old robot NPC that has a chance to give a good bullet)

rooms that require a key (keycard)

elite enemy room

multiple regions

Suggestions: "pacifist" option only for bosses (just kill normal enemies), which are more intelligent, interact via dialogue. After boss fight, will stop fighting, and player can decide to deal final blow or not.

For bosses, "eviler" option would be to kill and salvage fallen boss' parts.

Extra: "evil" pt tracker, each time do "evil" will make game harder
multi wave enemies

Devices:

Explain which input devices you plan on supporting and how they map to in-game controls.

Control	Input Device
Player movement	W / A / S / D
Player aiming	Mouse movement / cursor position
Player shooting	LMB
Player dash	SPACE or RMB
Player world interaction	E
Pause game / Open menu	ESC
Menu interaction	Mouse and LMB

Tools:

Specify and motivate the libraries and tools that you plan on using except for C/C++ and OpenGL.

Team management:

Identify how you will assign and track tasks and describe the internal deadlines and policies you will use to meet the goals of each milestone.

We will be using Trello to organize upcoming tasks and deadlines, as well as assign tasks to each group member. We will use a weekly sprint approach, ending on Monday of each week. On the same day, unfinished tasks are rolled over and new tasks are assigned.

Development Plan:

Provide a list of tasks that your team will work on for each of the weekly deadlines. Account for some testing time and potential delays, as well as describing alternative options (plan B). Include all the major features you plan on implementing (no code).

Milestone 1: Skeletal Game

Week 1 - player controls

- Systems design
- ECS system, main game loop
- OpenGL basics
- simple graphical representations of player, room, enemy (greyboxing)
- player movement controls (using WASD), dashes (space)
- player sprite movement (translation)
- some UI for player stack (basic feature 1)

Week 2 - bullets & collision

- collision detection with player & bullet, player & wall, player & enemy
- bullet stacking & stat modification system with at least 2 types of bullets
- player shooting towards cursor
- basic dialogue system for scientist to communicate with player
- bullet dealing damage to enemies/player
- Game over condition / screen
- Pause menu

Milestone 2: Minimal Playability

Week 1

- player animation (walking, shooting)
- bullet spawning systems
- basic enemy AI (walk around, shoot bullets at player)
- sprite work
- non-intrusive tutorial at start (skippable)
- ability to move onto the next room (make 2 rooms) - via procedurally linking or predetermined map layout
- Mesh-based Collision Detection (of non trivial shapes, so probably with a complex looking enemy) (from canvas milestone requirements)

Week 2

- sound system, with 1 main soundtrack and bullet shooting sound
- ability for player to interact with objects in room - for clearing stacks, story panels, chests etc.
- ability to hover over bullets in stack while game is paused
- 2 more enemy variants
- bouncing/piercing/homing bullets (optional in case of delays)

Milestone 3: Playability

Week 1

- add 2 more types of rooms (special rooms, boss rooms)
- add 1 boss (scientist)
- Lightning bullet / stack RNG system
- game ending cutscene (after scientist fight) + alternate ending

Week 2

- design variants of normal enemy rooms (same room size, same room shape, differ in appearance, enemies spawn formation / types)
- create 3 enemies

Milestone 4: Final Game

Week 1

- advanced enemies - BOIDs, worm AI
- Asset + sound cleanup / finalising
- Bug testing / fixing
- another region

Week 2

- another boss
- More enemy variants